

Araç Takip Simülasyonu

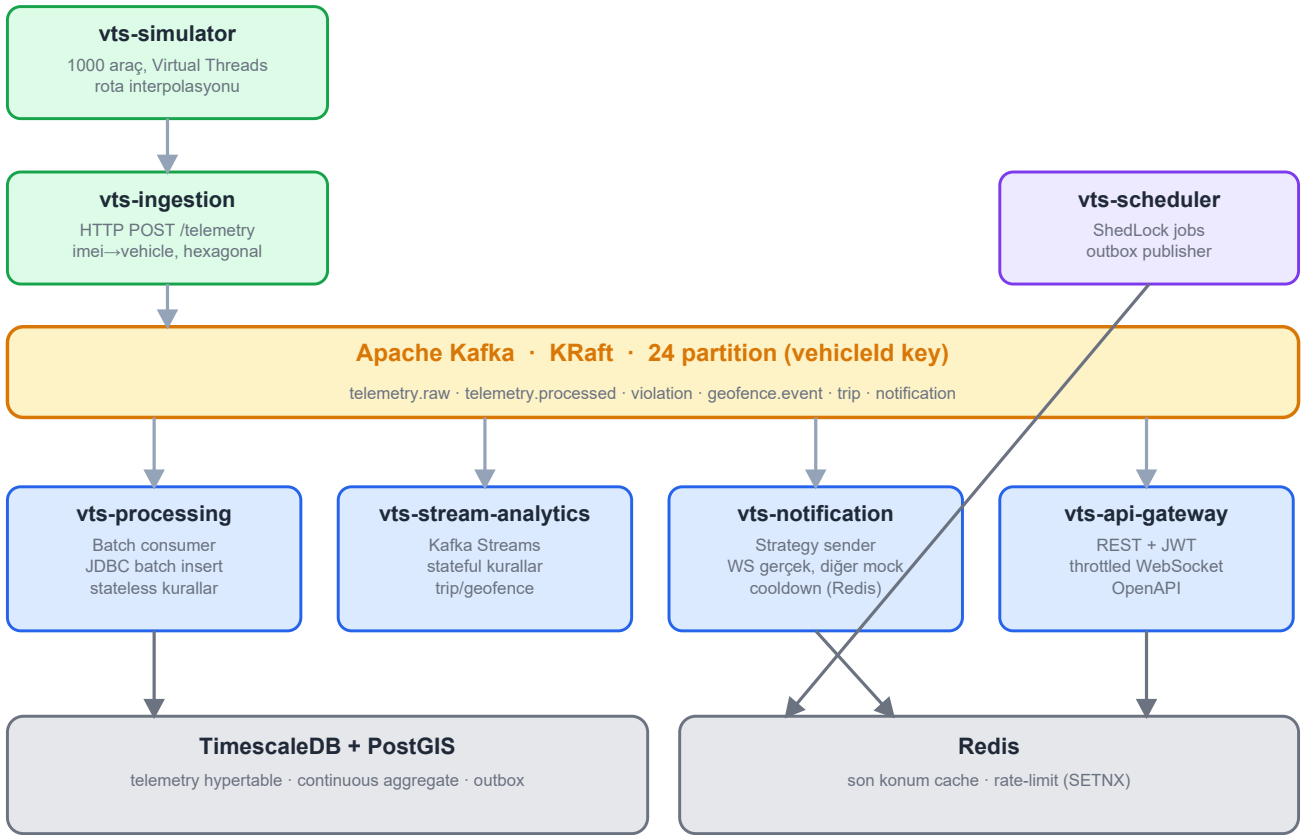
Olay Tabanlı Filo Telematik Platformu — Mimari Doküman

Java 21 · Spring Boot 3.3 · Apache Kafka · TimescaleDB · Çok modüllü Maven monorepo

1. Genel Bakış

Bu platform, simüle edilen araç cihazlarından gelen telemetriyi uçtan uca işleyen **olay tabanlı (event-driven)** bir filo takip sistemidir. Veri; **ingestion** → **Kafka** → **işleme/analitik** → **bildirim** → **API ağ geçidi** hattı boyunca akar. Tasarım hedefi **1000 araç, saniyede 1000 mesaj ve günde ~86 milyon satır**; çalışma (dev) profili ise **100 araç ve 5 sn tick** (~20 msg/sn). Bu ikisi çalışmaz: **ölçek yalnızca konfigürasyondan gelir**, mimari baştan doğru kurulur, sonradan yeniden yazım (rewrite) gerektirmez.

2. Uçtan Uca Akış



Şekil 1 — Yeşil: veri girişi · Turuncu: Kafka olay veri yolu · Mavi: tüketici servisler · Mor: zamanlanmış işler · Gri: paylaşımlı veri depoları

3. Modüller

Modül	Sorumluluk	Anahtar teknoloji
vts-common	Event modelleri, topic sabitleri, enum'lar, TenantContext	Java record, Jackson
vts-simulator	Filo simülatörü; rota üzerinde interpolasyon, kasıtlı anomaliler	Virtual Threads, REST
vts-ingestion	Stateless giriş; imei→vehicle lookup, doğrulama, Kafka publish	Hexagonal, Caffeine, Redis
vts-processing	Batch consumer; JDBC batch insert, stateless kurallar, outbox	Spring Kafka, JdbcTemplate

Modül	Sorumluluk	Anahtar teknoloji
vts-stream-analytic s	Stateful kurallar: sert fren, sürekli hız, rölanti, geofence, trip	Kafka Streams, RocksDB
vts-notification	Bildirim gönderimi; Strategy pattern, cooldown, quiet hours	WebSocket, Redis SETNX
vts-api-gateway	REST + JWT; canlı harita WebSocket (delta+viewport); şema sahibi	Spring Security, JPA, Flyway
vts-scheduler	Cihaz offline, skarlama, bakım, outbox publisher	ShedLock (Postgres)

4. Ölçek Kısıtları ve Gerekçeleri

Aşağıdaki kararlar “sonra optimize ederiz” denemeyecek, baştan doğru kurulması gereken şeylerdir:

Kısıt	Gerekçe
Telemetri tekil save() ile yazılmaz	Batch Kafka consumer + JdbcTemplate.batchUpdate() + ON CONFLICT DO NOTHING. Telemetri için JPA entity yok — Hibernate dirty checking 1000/sn'de CPU yer. JDBC'de reWriteBatchedInserts=true.
Event başına Redis round-trip yok	Stateful kurallar Kafka Streams state store (RocksDB) ile tutulur; toplu Redis işlemleri pipeline ile yapılır.
WebSocket'e event başına mesaj yok	Gateway in-memory tutar, @Scheduled ile saniyede 1 kez SADECE değişenleri (delta) yayımlar; client viewport (bbox) gönderir, sadece o alan iletilir.
Kafka partition = 24 (sabit)	Profilden bağımsız. Sonradan artırmak vehicleId hash dağılımını değiştirir, per-vehicle ordering'i kırar, Streams state store'larını bozar.
Telemetri = TimescaleDB hypertable	Şişmiş normal tabloyu sonradan hypertable'a çevirmek downtime demektir. Dashboard sorguları ham tabloya değil continuous aggregate'e vurur.
Her tabloda tenant_id + Outbox	38 tabloya sonradan kolon eklemek migration cehennemidir; çok kiracılık ve olay atomikliği baştan kurulur.

5. Veri Modeli (TimescaleDB)

Şema **Flyway V1–V15** ile yönetilir ve **38 iş tablosu** içerir (kimlik/kiracı, filo, telemetri, kurallar, geofence, seferler, bildirimler, bakım, skorlama, altyapı). Kritik noktalar:

- **telemetry** hypertable: zaman (ts) + hash (vehicle_id, 8 partition); PK (vehicle_id, ts) ingestion'a temiz ON CONFLICT hedefi verir; FK yok (batch insert hızı).
- **violation** hypertable: olay üretim hızına yakın; okuma modeli olarak entity'lenir.
- **vehicle_driver_assignment**: ihlali doğru şoföre atfetmek için zamansal kayıt (vehicle.driver_id tek başına yetmez).
- **Continuous aggregate'ler**: telemetry_1min, telemetry_hourly, violation_daily_summary — dashboard bunlara vurur.
- **Kompresyon + retention** politikaları ve GIST/BRIN/partial index'ler baştan tanımlı.

Doğrulama: tüm 15 migration canlı *timescaledb-ha:pg16* üzerinde temiz uygulandı; 36 JPA entity *ddl-auto=validate* ile şemaya birebir eşleşti (Testcontainers/entegrasyon testi).

6. Kafka Topic Tasarımı

Topic	Üretici → Tüketici	Not
vehicle.telemetry.raw	ingestion → processing, stream-analytics	key=vehicleId
vehicle.telemetry.processed	processing → gateway	UI fan-out
vehicle.violation	processing/analytics → notification	ihlaller
vehicle.geofence.event	analytics → notification	giriş/çıkış
vehicle.trip	analytics → gateway	sefer kapanışı
.dlq / .retry-5s / .retry-1m	hata zinciri	exponential backoff

7. Kural Motoru

Eşik değerleri **hard-code değildir**; rule + rule_assignment tablosundan okunur ve Caffeine ile lokal cache'lenir (TTL 60 sn), değişiklikte Kafka üzerinden cache invalidation yayılır. **Stateless** kurallar (hız limiti, düşük

batarya/yakıt) processing-service'te Strategy pattern ile; **stateful** kurallar (sert fren, sürekli hız, rölanti, geofence, trip) Kafka Streams topolojisinde çalışır. Kapsam (scope): kural TENANT/GROUP/VEHICLE düzeyinde override edilebilir — örn. tıra 80, otomobile 110 km/s.

8. Gözlemlenebilirlik & Güvenlik

Observability baştan kuruludur: Micrometer + Prometheus + Grafana; consumer lag, ingested/persisted, violation.produced, batch.insert.duration, e2e.latency metrikleri; her olayda correlationId ile yapılandırılmış JSON log. **Güvenlik**: Spring Security + JWT; roller ADMIN, FLEET_MANAGER, DRIVER, VIEWER; çok kiracılık Hibernate @Filter ile izole edilir.

9. Teknoloji Yığını

Katman	Teknoloji
Dil / Runtime	Java 21 (Virtual Threads), Spring Boot 3.3.5
Mesajlaşma	Apache Kafka (KRaft), Kafka Streams
Veritabanı	PostgreSQL 16 + TimescaleDB + PostGIS
Cache	Redis, Caffeine (lokal)
Persistence	Hibernate 6 / JPA, JdbcTemplate (telemetri), Flyway, MapStruct
Güvenlik	Spring Security, JWT (OAuth2 resource server)
Gözlemlenebilirlik	Micrometer, Prometheus, Grafana, logstash JSON
Test / Altyapı	JUnit 5, Testcontainers, Docker Compose

10. Profiller

Profil	Araç	Tick	Chunk	Retention
dev (varsayılan)	100	5 sn	1 gün	30 gün
load	1000	1 sn	1 saat	7 gün
prod	dış konfig	—	—	—

11. Mevcut Durum

Adım	İçerik	Durum
1	Monorepo iskeleti, 8 modül, docker-compose altyapısı	Tamam ☐
2	Flyway V1–V15 (38 tablo) — canlı DB'de doğrulandı	Tamam ☐
3	vts-common: event modelleri, topic sabitleri, enum'lar	Tamam ☐
4	36 JPA entity + repository — ddl-auto=validate geçti	Tamam ☐
5	vts-ingestion (hexagonal HTTP, imei→vehicle, Kafka, DLQ)	Tamam ☐
6	vts-simulator (Virtual Threads, rota interpolasyonu, anomaliler)	Tamam ☐
7	vts-processing (batch insert, kurallar, outbox) — uçtan uca test edildi	Tamam ☐
8–14	Stream-analytics, bildirim, gateway, scheduler, gözlemlenebilirlik, testler, README	Sırada

Uçtan uca doğrulama (Adım 7 sonunda): simulator → ingestion → Kafka → processing → TimescaleDB zinciri gerçek konteynerlerde çalıştırıldı. Telemetri hypertable'a yazıldı, vehicle_last_position UPSERT edildi, hız ihlalleri + outbox üretildi. GROUP eşik override'i kanıtlandı: TIR (limit 80) ihlal etti, otomobil (limit 110) 95-100 km/s'de etmedi; ihlaller doğru şoföre atfedildi.

12. Çalıştırma

Altyapıyı ayağa kaldırma (tek komut):

`docker compose up -d` → Kafka + Kafka UI (:8090), TimescaleDB (:5432), Redis (:6379), Prometheus (:9090), Grafana (:3000). kafka-init tüm topic'leri 24 partition ile açar. Yük profili için `-f docker-compose.yml -f docker-compose.load.yml`.

Depo: github.com/AllenVB/Vehicle-Tracking-Simulation